

Ledger Financeira

Saques concorrentes numa ledger em PostgreSQL,
feitos por threads sem lock nem sincronismo.

Sumário

-
- | | | |
|----|----------|---------------------------------------|
| 01 | A ledger | O modelo de dados e um saque de 50,00 |
|----|----------|---------------------------------------|
-
- | | | |
|----|-----------------------|--------------------------------------|
| 02 | A condição de corrida | Ler, processar, escrever, intercalar |
|----|-----------------------|--------------------------------------|
-
- | | | |
|----|------------|--|
| 03 | Resultados | Seis casos, do sequencial ao heisenbug |
|----|------------|--|
-
- | | | |
|----|-------------|--|
| 04 | Comparações | Perda, tempo e ganho de 1 a 10 threads |
|----|-------------|--|
-

01

A ledger

O que é uma ledger

Um livro-razão

A ledger registra cada movimento de dinheiro como uma linha, com a conta, o valor e o horário.

Só se escreve no fim

As linhas não são editadas nem apagadas: um estorno entra como uma nova linha de sinal contrário.

O saldo é derivado

O saldo da conta é a soma dos lançamentos. A coluna saldo em contas é um cache desse total, e é ela que as threads corrompem.

A ledger no Postgres

CONTAS

ID	TITULAR	SALDO
1	Matheus	100,00
2	João Vitor	340,00

LANCAMENTOS

ID	CONTA	VALOR	CRIADO_EM
17	1	+150,00	09:12:04
18	1	-50,00	09:41:22

Invariante da ledger: *contas.saldo* é igual à soma de *lancamentos.valor* da conta. Aqui, 150,00 – 50,00 = 100,00.

Um saque de 50,00, linha por linha

Antes

Conta 1 tem 100,00

```
SELECT saldo FROM contas
WHERE id = 1;
-- 100,00
```

Lançamento

Registra o débito

```
INSERT INTO lancamentos
  (conta, valor)
VALUES (1, -50.00);
```

Depois

Atualiza o saldo

```
UPDATE contas
SET saldo = 50.00
WHERE id = 1;
```

As duas escritas ficam na mesma transação, então o INSERT e o UPDATE entram juntos ou nenhum entra.

02

A condição de corrida

Leitura, processamento, escrita

Etapa 1 · leitura

Lê o saldo

```
SELECT saldo FROM contas  
WHERE id = 1;
```

Etapa 2 · processamento

Decide na aplicação

```
if (saldo >= valor)  
    novo = saldo - valor;
```

Etapa 3 · escrita

Grava o valor calculado

```
UPDATE contas  
SET saldo = novo  
WHERE id = 1;
```

A etapa 2 decide com o número lido na etapa 1 e a etapa 3 grava esse resultado. Entre o SELECT e o UPDATE passam alguns milissegundos, e nesse intervalo outra thread lê o mesmo saldo.

O cenário do experimento

CONTA 1

100,00

Saldo inicial, uma única conta no banco.

THREADS

2

Duas threads disparadas ao mesmo tempo, sem lock.

SAQUE DE CADA UMA

80,00

Cada thread verifica o saldo antes de sacar.

Em execução sequencial, o segundo saque seria recusado por saldo insuficiente.

Como as duas threads se intercalam

T	THREAD A	THREAD B	SALDO NO BANCO
t1	lê saldo → 100,00		100,00
t2		lê saldo → 100,00	100,00
t3	100 ≥ 80 → grava 20,00		20,00
t4		100 ≥ 80 → grava 20,00	20,00

A thread B calcula com o saldo lido em t2 e sobrescreve o UPDATE de A. É o *lost update*.

80,00

Saíram 160,00 de uma conta com 100,00 e o saldo terminou em 20,00. Faltam 80,00 entre a coluna saldo e a soma dos lançamentos, e o Postgres não reporta erro nenhum.

03

Resultados

A máquina que produziu os números

Ambiente de medição

CPU	Apple M4
Memória	16 GB
Sistema	darwin/arm64
Node	v26.8.0-alpha
Banco	Postgres 17 em container local
Medição	2026-09-07, 10 repetições por ponto, warm-up descartado
Varredura	1, 2, 4, 8, 10, 16 e 20 threads

Threads de hardware

10

Esse número explica onde as curvas viram. Todo gráfico traz uma régua vermelha vertical em 10 threads, e à direita dela os workers se revezam nos mesmos dez fios em vez de rodar em paralelo. Em uma máquina com outra contagem de núcleos a régua muda de lugar e os gráficos mudam de forma.

Caso 01 · Baseline sequencial

O que faz

Um saque por vez, em laço, na thread principal: SELECT, cálculo em JS, UPDATE. Nenhum worker sobe.

Por que existe

Valida a lógica do saque e fixa o tempo de referência contra o qual o caso 04 é lido.

Divergência zero, porque cada SELECT já enxerga o UPDATE anterior.

Resultado

THREADS	SAQUES	TEMPO	PERDA
1	200	98 ms	0%

Caso 02 · CPU dividida entre threads

O que faz

Um laço de mistura de inteiros, puro CPU, dividido em partes iguais entre N workers. Nenhum estado compartilhado e nenhum banco.

De 1 para 10 threads o tempo cai 4,76x, e acima do limite da máquina volta a subir. A perda fica em 0% em todos os pontos, e é esse zero que separar o trabalho compra.

Resultado

THREADS	TEMPO	PERDA
1	1060 ms	0%
2	544 ms	0%
4	318 ms	0%
10	223 ms	0%
20	298 ms	0%

Caso 03 · Corrida em SharedArrayBuffer

O que faz

N workers incrementam o mesmo Int32Array, sem Atomics.

O bug

contador[0] = contador[0] + 1 são três operações de máquina: carrega, soma, escreve. Dois núcleos carregam o mesmo valor, escrevem o mesmo resultado e um dos incrementos desaparece.

Com 1 thread a perda é 0%, porque não há com quem disputar. De 2 threads em diante ela passa de 40% em todos os pontos.

Resultado

THREADS	TEMPO	PERDA
1	53 ms	0%
2	47 ms	48%
4	40 ms	73%
10	75 ms	56%
20	160 ms	64%

Caso 04 · Corrida no banco

O que faz

O saque do caso 01, agora executado por N workers na mesma conta, cada um com o próprio Pool.

O bug

Entre o SELECT e o UPDATE outra thread já leu o mesmo saldo velho, e a segunda gravação apaga a primeira.

A ledger registra os 200 saques e a conta debita um terço deles. O tempo piora a partir de 2 threads: disputa pela mesma linha não paraleliza.

Resultado

THREADS	TEMPO	PERDA
1	164 ms	0%
2	133 ms	38%
4	138 ms	54%
10	318 ms	67%
20	442 ms	66%

Caso 05 · Leitura suja

O que faz

N workers transferem entre pares exclusivos de contas, sem transação, enquanto a thread principal roda SELECT SUM(saldo) em laço, como faria um dashboard.

No fim nenhum centavo se perde, e o relatório mostrou totais que nunca foram verdade. Cada transferência no ar esconde 100 unidades, então o buraco cresce junto com o número de threads.

Resultado

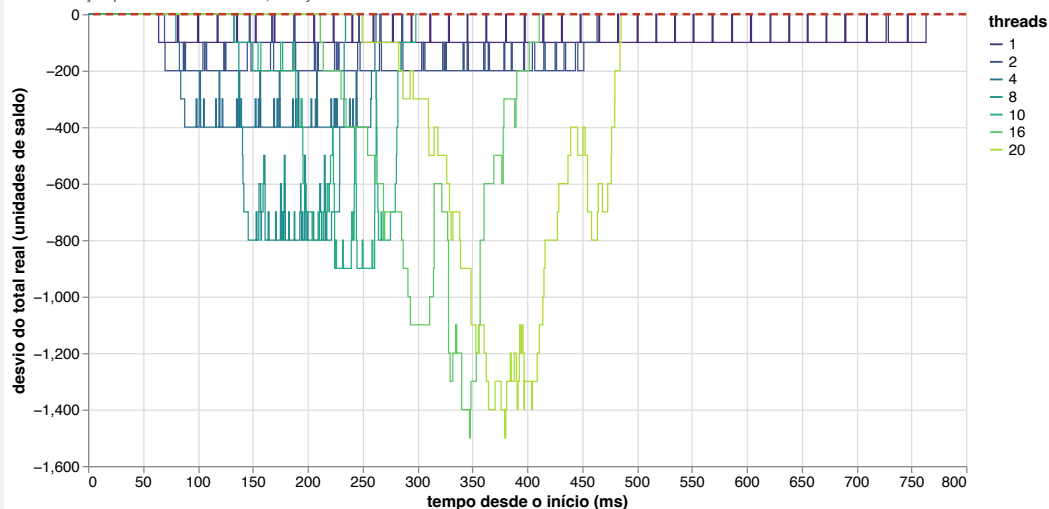
THREADS	TEMPO	PERDA FINAL	MENOR TOTAL
1	778 ms	0%	-100
2	440 ms	0%	-200
4	300 ms	0%	-400
10	267 ms	0%	-900
20	499 ms	0%	-1500

O dashboard leu totais que nunca existiram

SELECT SUM(saldo) durante transferências sem transação

Apple M4 · 10 threads de hardware · node v26.8.0-alpha.0.0.0

uma execução por número de threads, eixo y como desvio do total real



Caso 06 · O heisenbug

O que faz

O caso 03 inteiro, com uma única diferença: uma flag insere um console.error dentro da seção crítica, entre ler e escrever o Int32Array.

Com 2 threads o log derruba a perda de 29% para 2,4%. A partir de 8 threads a comparação se inverte. O log não toca na lógica, só muda o tempo, e a corrida depende de duas threads caírem na mesma janela. Como ninguém sabe de cabeça o tamanho dessa janela, um print nunca é prova de nada.

Resultado

THREADS	PERDA SEM LOG	PERDA COM LOG
1	0%	0%
2	29%	2,4%
4	17%	9,2%
10	7,1%	12,8%
20	5,2%	6,3%

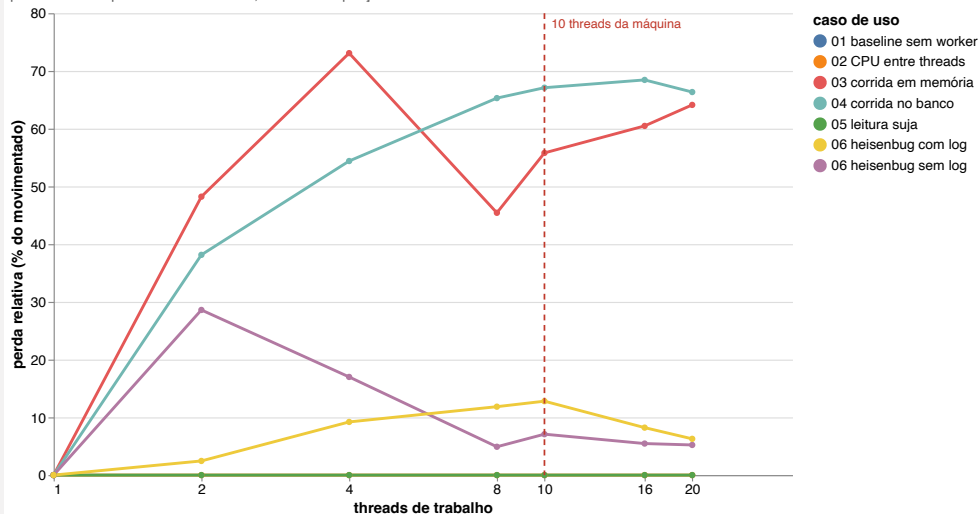
04

Comparações

Quem compartilha estado perde de 40% a 70%

Perda relativa por número de threads

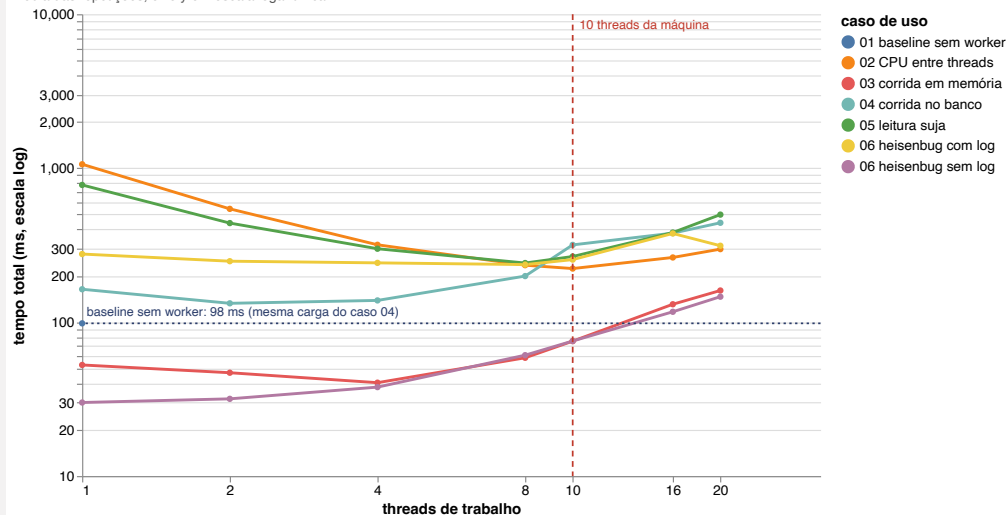
Apple M4 · 10 threads de hardware · node v26.8.0-alpha.0.0.0
perdido dividido pelo total movimentado, média das repetições



Depois de 10 threads, cada worker piora o tempo

Tempo por número de threads

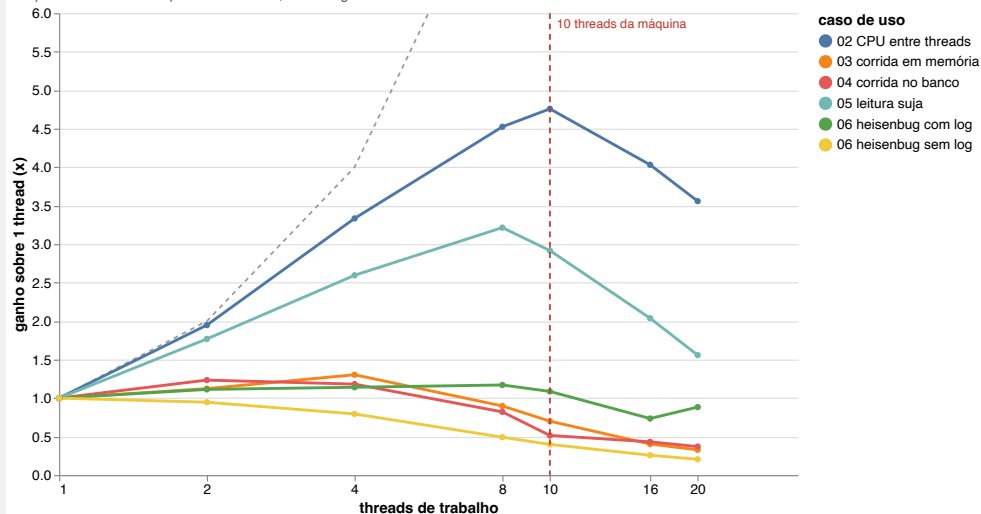
Apple M4 · 10 threads de hardware · node v26.8.0-alpha.0.0.0
média das repetições, eixo y em escala logarítmica



Três casos ficam abaixo de 1x de ganho

Ganho de velocidade por número de threads

Apple M4 · 10 threads de hardware · node v26.8.0-alpha.0.0.0
tempo com 1 thread / tempo com N threads, contra o ganho ideal



Thread acelera trabalho que pode ser separado, como no caso 02, e cobra caro em trabalho que disputa o mesmo estado, como nos casos 03, 04 e 06, onde piora o tempo e ainda destrói o dado. O limite de threads da máquina é o ponto onde a conta vira.